centralelille

Département
de
Mathématiques et Informatique

Système et réseau : module n° 2

Séminaire n° 5

Threads, mutex et sémaphores

Samir El Khattabi *et* Christian Vercauter

Version éditée le 8 avril 2020

Année 2019-2020

Table des matières

Introduction **4**

I Mutex **5**

1 Motivation . . . 5
1.1 Énoncé du problème . . . 5
1.2 Spécifications détaillées . . . 5
1.3 Présentation d'une solution . . . 6
1.4 Test de cette solution . . . 8
1.5 Ressource critique et section critique . . . 9
2 Mutex . . . 10
2.1 Définition . . . 10
2.2 Propriétés . . . 10
2.3 Créer et initialiser un mutex . . . 11
2.4 Verrouiller et déverrouiller . . . 11
2.5 Supprimer un mutex . . . 12
3 Exemple d'application . . . 12
3.1 Objectif . . . 12
3.2 Solution . . . 12

II Sémaphores **13**

1 Définition et propriétés . . . 13
2 Sémaphores Posix . . . 13
2.1 Fichier d'en-tête, type et librairie . . . 14
3 Opérations sur sémaphores . . . 14
3.1 Opérations spécifiques sur sémaphores nommés . . . 15
3.2 Opérations spécifiques sur sémaphores anonymes . . . 17
3.3 Opérations communes aux deux types de sémaphores . . . 18
4 Exercice . . . 19
5 Sémaphores vs mutex pthread . . . 19

III Exercice de synthèse **20**

1 Présentation et objectifs . . . 20
1.1 Objectif . . . 20
1.2 Spécifications détaillées . . . 20
2 Un rédacteur et un lecteur . . . 22
2.1 Test de la version initiale . . . 22
2.2 Accès concurrent au buffer circulaire . . . 23

3 Un rédacteur et m lecteurs 24
3.1 Spécifications 24
3.2 Modifications à apporter 24
3.3 Tests 25
4 n rédacteurs et m lecteurs 25
4.1 Spécifications 25
4.2 Modifications à apporter 25
4.3 Tests 26

Introduction

Ce document est un recueil d'exercices complété de rappels de cours, qui a été réalisé à partir de polycopiés de cours et d'exercices créés par Étienne Craye, Samir El Khattabi et Christian Vercauter pour plusieurs enseignements effectués à EC-Lille et à IG2I Lens.

Il aborde quelques mécanismes de coopération et de synchronisation entre threads :

- les sémaphores d'exclusion mutuelle, ou *mutex*
- les sémaphores à compte

Les informations présentées ici, sont conformes aux standards `POSIX` de l'IEEE établis à partir de la fin des années 1980, complétés et périodiquement révisés depuis ; la dernière révision date de 2017 (IEEE Std 1003.1™-2017)

Les exemples pris et les exercices proposés ont été testés sur un système `Linux` installé sur un ordinateur d'architecture `x86-64` bits.

Ils font appel à :

- des compétences SHELL acquises dans le module 1 de cet électif ;
- des connaissances de base sur la programmation en langage C, qui seront enrichies lors de cette séance, par l'usage important de pointeurs de toutes sortes ;
- des compétences sur les outils de développement associés.



Le compilateur `Gcc`



L'éditeur de textes `Emacs`



L'éditeur de textes `vim`

Dans la version avec réponses aux questions, de ce document, les solutions présentées ont été obtenues à partir de plusieurs ordinateurs dans différentes configurations :



`Ubuntu 18.04 LTS` en mode virtuel avec `VirtualBox`



`Ubuntu 18.04 LTS` en mode natif



`Debian 8.10` en mode natif

Chapitre I

Mutex

1. Motivation

1.1 Énoncé du problème

Un processus (*thread principal*) crée deux threads chargés de traiter un même message.

Le traitement du message consiste simplement à afficher son contenu élément par élément (caractère par caractère). On supposera que le traitement d'un élément prend un temps variable dont la durée sera obtenue à l'aide d'un générateur de nombres pseudo-aléatoires ; cela signifie que ce n'est pas forcément le thread qui démarre en premier sa tâche d'affichage, qui la termine en premier.

On considérera également que chaque thread, une fois créé, met un temps variable, généré de façon aléatoire pour se préparer à la tâche qui lui incombe ; ceci signifie que le thread créé en premier ne sera peut-être pas le premier à démarrer sa tâche.

1.2 Spécifications détaillées

1. La durée de préparation [1] d'un thread est comprise entre 100 et 400 ms ;
2. la durée de traitement d'un élément est compris entre 50 et 200 ms ;
3. le premier thread créé affiche les caractères du message en minuscules ;
4. le second thread créé affiche les caractères du message en majuscules ;
5. Chaque thread enfant construit un nombre réel à partir du nombre de caractères de chaque mot du message à traiter ; un mot est une séquence de caractères alphabétiques. Ce nombre ainsi obtenu définit le code de terminaison du thread enfant.

   Si le message traité est « ***ME**s **V**ieilles **T**ortues **MA**rchent **J**ustement **S**ur **U**n **N**inja* » [2] alors la valeur obtenue est 3.8789325.
6. le thread principal affiche son *pid* et son *tid* avant de créer les deux threads enfants, puis attend leur terminaison. Il affiche ensuite leur code de terminaison et se termine en affichant à nouveau son *pid* et son *tid*, comme dans l'extrait ci-dessous :

   ```
   $ ./mutexThread0
   Le processus (pid = 24539, tid = 0x7f848a08f740) va créer 2 threads
   Attente de la terminaison des 2 threads
   ```

1. intervalle entre l'instant de création et le début effectif du traitement
2. MErcure, Vénus, Terre, MArs, Jupiter, Saturne, Uranus et Neptune

```
...
Fin du thread n°1 avec le code 3.1415926535

Fin du thread n°2 avec le code 3.1415926535
Fin du processus 24539 (tid = 0x7f848a08f740)
$
```

1.3 Présentation d'une solution

Le code ci-dessous est basé sur celui présenté dans le document de la séance précédente :

```c
/* ---------------------------------------------------------------------- */
/*                  E N T Ê T E S   S T A N D A R D S                     */
/* ---------------------------------------------------------------------- */
#include <stdio.h>
#include <stdlib.h>
#include <ctype.h>
#include <time.h>

#include <unistd.h>
#include <pthread.h>

/* ---------------------------------------------------------------------- */
/*             C O N S T A N T E S   S Y M B O L I Q U E S                */
/* ---------------------------------------------------------------------- */
#define NBMAX_THREADS   2
#define TEMPO_10ms      10000000L

/* ---------------------------------------------------------------------- */
/*             D É F I N I T I O N S   D E   T Y P E S                    */
/* ---------------------------------------------------------------------- */
typedef void * (*pf_t)(void *);

/* ---------------------------------------------------------------------- */
/*                     M A C R O - F O N C T I O N S                      */
/* ---------------------------------------------------------------------- */
#define CHECK_T(status, msg)                            \
  if (0 != (status))   {                                \
    fprintf(stderr, "pthread erreur : %s\n", msg);      \
    exit (EXIT_FAILURE);                                \
  }

/* ---------------------------------------------------------------------- */
/*           P R O T O T Y P E S   D E   F O N C T I O N S                */
/* ---------------------------------------------------------------------- */
void monThread (long no);
void bye(void);

/* ---------------------------------------------------------------------- */
/*                V A R I A B L E S   G L O B A L E S                     */
/* ---------------------------------------------------------------------- */
const char *msgPi = "Que J'aime A Faire Apprendre Un NOMBRE Utile Aux SAGES";

/* ---------------------------------------------------------------------- */
int main(int argc, char *argv[])
{
    pthread_t tid[NBMAX_THREADS];
    long i;
    double * status;

    srand(time(NULL));  /* Init. du générateur de nombres pseudo-aléatoires */
    atexit(bye);
    printf("Le processus (pid = %d, tid = 0x%lx) va créer %d threads\n",
           getpid(), (long)pthread_self(), NBMAX_THREADS);
    /* Création des NBMAX_THREADS                                          */
    for (i = 0; i < NBMAX_THREADS; i++)
        CHECK_T(pthread_create (&tid[i], NULL, (pf_t)monThread,
                                (void *)i), "pthread_create()");
```

```
58
59      printf("Attente de la terminaison des %d threads\n\n", NBMAX_THREADS);
60      for (i = 0; i < NBMAX_THREADS; i++) {
61          CHECK_T(pthread_join (tid[i], (void **) & status),"pthread_join()");
62          printf("\nFin du thread n°%ld avec le code %.10f\n", i + 1, *status);
63          free(status);
64      }
65      return EXIT_SUCCESS;
66  }
67
68  /* ------------------------------------------------------------------------ */
69  void  monThread (long  no)
70  {
71      double * retVal = (double *) calloc (1, sizeof(double));
72      struct timespec delai = { .tv_sec = 0, .tv_nsec = 0};
73      long n = 0;
74      double diviseur = 1.;
75      const char *p;
76      /*  Attente initiale du thread                                          */
77      delai.tv_nsec = TEMPO_10ms * (10 + rand() % 31);
78      nanosleep(&delai, NULL);
79
80      for (p = msgPi; *p != '\0'; p++) {
81          if (isalpha(*p)) {
82              n++;
83              if (no == 1)
84                  printf("%c", tolower(*p));
85              else
86                  printf("%c", toupper(*p));
87          } else {
88              if (n > 0) {
89                  *retVal += n / diviseur;
90                  n = 0;
91                  diviseur *= 10.;
92              }
93              printf("%c", *p);
94              fflush(stdout);
95          }
96          /* Attente entre le traitement de chaque caractère                 */
97          delai.tv_nsec = TEMPO_10ms * (5 + rand() % 16);
98          nanosleep(&delai, NULL);
99      }
100     /* traitement du dernier mot                                           */
101     if (n > 0) *retVal += n / diviseur;
102
103     /* terminaison du thread                                               */
104     pthread_exit((void *) retVal);
105 }
106
107 /* ------------------------------------------------------------------------ */
108 void bye (void)
109 {
110     printf("Fin du processus %d (tid = 0x%lx)\n",
111            getpid(), pthread_self());
112 }
113 /* ------------------------------------------------------------------------ */
```

Listing I.1 – Le programme `mutexThread0.c`

On note dans cet exemple :

- la définition de la fonction qui implémente le corps des threads enfants : pas de résultat retourné et un paramètre qui est un entier long (ligne 35 pour le prototype et 69 pour la définition)

  Son utilisation lors de la création des threads enfants (ligne 56) nécessite un forçage de type effectué avec le type `pf_t`, comme présenté dans le document de la séance précédente.

- l'utilisation sur les lignes 78 et 98, de l'appel système `nanosleep()` pour obtenir des durées d'attente pouvant être inférieures à la seconde (cf. `man 2 nanolseep`)

- l'usage de macro-fonctions définies dans `ctype.h` pour le test de la nature de chaque caractère et leur conversion en minuscules ou en majuscules (lignes 84, 84 et 86)
- Le message à afficher est une chaîne de caractères constante (ligne 41) La parcourir avec un pointeur nécessite qu'il soit déclaré de type `const char *` (ligne 75)
- l'allocation de mémoire effectuée à l'aide de la fonction `calloc()` qui a la particularité d'initialiser à zéro, l'espace mémoire créé. (ligne 71)
- L'affichage sur l'écran est bufferisé — les caractères à afficher sont temporairement stockés dans un buffer, jusqu'à ce qu'y soit stocké le caractère `'\n'`, ce qui provoque l'affichage (le *vidage*) du buffer sur l'écran —.

  On provoque ce *vidage* à l'aide de `fflush(stdout);`

1.4 Test de cette solution

Le programme décrit précédemment est disponible sur la plate-forme Moodle de cet enseignement.

a) Compilation

S'agissant d'un programme utilisant des threads et des fonctions qui les manipulent, l'édition de liens doit être réalisée avec l'option `-lpthread`, comme dans l'exemple ci-dessous :

```
gcc mutexThread0.c -Wall -o mutexThread0 -lpthread
```

b) Test

L'exécution de ce programme donne le résultat présenté ci-dessous :

```
$ ./mutexThread0
Le processus (pid = 24787, tid = 0x7fee72e4f740) va créer 2 threads
Attente de la terminaison des 2 threads

QquUEe  Jj'A'aIiMmeE  Aa  FAfIRaiE rAe PPapRprENenDdRrEe U Nu n NOnMoBmRbE reU
TIuLtEi lAe UaXux  SAsaGgEeS
Fin du thread n°1 avec le code 3.1415926535
s
Fin du thread n°2 avec le code 3.1415926535
Fin du processus 24787 (tid = 0x7fee72e4f740)
$
```

On note que les résultats obtenus après terminaison des threads enfants sont bien ceux attendus : le nombre π avec ses dix premières décimales.
C'est le second thread créé qui a commencé l'affichage ; cela commence par la lettre `'Q'` mais c'est le premier thread créé qui se termine en dernier en affichant le dernier `'s'` du message.
Cependant les messages affichés sont difficilement déchiffrables car c'est un mélange de caractères affichés en minuscules par le thread n° 1 et de caractères affichés en majuscules par le thread n° 2.
On vérifie que deux exécutions de ce programme produisent des résultats différents en termes d'affichage de message comme le montre le résultat ci-dessous :

```
$ ./mutexThread0
Le processus (pid = 25075, tid = 0x7fd5b1d04740) va créer 2 threads
Attente de la terminaison des 2 threads
```

```
qQuUe Ej 'J'aAimIMe aE  fAa iFrAIe RaE ppAPrPeRnEdNrDReE  UuNn  nNOoMmBRbEr
Ue uTtIiLleE  aAuUxX  sSagAGEeSs
Fin du thread n°1 avec le code 3.1415926535

Fin du thread n°2 avec le code 3.1415926535
Fin du processus 25075 (tid = 0x7fd5b1d04740)
$
```

On se rend compte dans ce second exemple que le thread 1 a commencé l'affichage en premier mais l'a terminé en dernier.

1.5 Ressource critique et section critique

Le problème observé lors des tests précédents, est qu'il est difficile, voire impossible, de déchiffrer les messages affichés : il suffit d'imaginer que les messages affichés par chaque thread soient différents et qu'il n'y ait pas de conversions en minuscules ou en majuscules effectuées par ces threads.
Ces deux threads utilisent une même ressource : l'écran du terminal où s'affiche leur message. Pour éviter l'entrelacement des caractères affichés par l'un et par l'autre, il faudrait que le premier thread qui commence son affichage puis le faire jusqu'à la fin, avant que le second thread ne démarre le sien.
L'écran — et le code permettant son usage — est une *ressource critique* qui ne peut être détenue à un instant donné, que par un seul propriétaire.
Une ressource critique est donc une ressource partagée par plusieurs processus ou threads, dont il faut garantir qu'à tout instant, elle n'ait qu'un seul possesseur.
Cela peut être, par exemples :

- une ressource matérielle, comme dans l'exemple précédent ;
- une structure de données ;
- un fichier ;
- une table d'une base de données, …

La section d'un programme où est utilisée une ressource critique constitue la *section critique*.
Dans l'exemple précédent, les opérations ci-dessous forment une section critique

```
    for (p = msgPi; *p != '\0'; p++) {
        if (isalpha(*p)) {
            n++;
            if (no == 1)
                printf("%c", tolower(*p));
            else
                printf("%c", toupper(*p));
        } else {
            if (n > 0) {
                *retVal += n / diviseur;
                n = 0;
                diviseur *= 10.;
            }
            printf("%c", *p);
            fflush(stdout);
        }
        /* Attente entre le traitement de chaque caractère                    */
        delai.tv_nsec = TEMPO_10ms * (5 + rand() % 16);
        nanosleep(&delai, NULL);
    }
    /* traitement du dernier mot                                              */
```

Listing I.2 – Section critique du programme `mutexThread0.c`

2. Mutex

2.1 Définition

Un mutex est un objet d'exclusion mutuelle (**MUT**ual **EX**clusion), utilisé pour protéger l'accès *simultané* à une ressource critique.
Un introduction aux threads Posix est disponible dans les manuels UNIX à l'aide de la commande `man 7 pthreads`

2.2 Propriétés

a) État

Un mutex peut être dans deux états :

déverrouillé : il n'est pris par aucun thread ;

verrouillé : il est temporairement la propriété d'un thread.

Un mutex ne peut être pris que par un seul thread à la fois. Lorsqu'un thread demande à verrouiller un mutex déjà verrouillé, celui-ci est suspendu, jusqu'à ce que le mutex soit libéré.

b) File des threads en attente

Un mutex est donc également caractérisé par l'identité de son propriétaire à l'instant présent et par une liste de threads en attente de sa disponibilité.

c) Type

Outre son état, l'identifiant de son propriétaire à un instant donné et la liste des identifiants des threads en attente, un mutex a également un type qui détermine le comportement du thread qui demande à le verrouiller alors qu'il est déjà verrouillé.

- Si le mutex est de type *rapide* alors le thread qui demande le verrouillage d'un mutex occupé est suspendu ; c'est le comportement par défaut ;
- Si le mutex est de type avec *vérification d'erreur*, alors la demande de verrouillage est immédiatement interrompue et retourne un code d'erreur `EDEADLK` ;
- Si le mutex est de type *récursif*, une demande de verrouillage du thread propriétaire est immédiatement acceptée. Ce thread doit alors effectuer autant d'opérations de déverrouillage que de verrouillages, pour libérer le mutex.

Le type de mutex par défaut est *rapide*, c'est-à-dire `PTHREAD_MUTEX_FAST_NP`.

2.3 Créer et initialiser un mutex

La librairie Posix des threads a défini le type `pthread_mutex_t` pour déclarer de tels objets dans des programmes, comme dans l'exemple ci-dessous :

```
pthread_mutex_t monMutex;
```

De telles variables destinées à être utilisées par plusieurs threads c.-à-d. dans plusieurs fonctions qui implémentent le corps de ces threads, sont généralement définies de façon globale, par le thread principal.

a) Initialisation statique

De plus, une variable de type `pthread_mutex_t` peut être initialisée de manière statique lors de leur déclaration, en utilisant les constantes …

- `PTHREAD_MUTEX_INITIALIZER`, pour les mutex *rapides*
- `PTHREAD_RECURSIVE_MUTEX_INITIALIZER_NP`, pour les mutex *récursifs*
- `PTHREAD_ERRORCHECK_MUTEX_INITIALIZER_NP`, pour les mutex à *vérification d'erreur*

… comme dans les exemples ci-dessous :

```
pthread_mutex_t fastMutex = PTHREAD_MUTEX_INITIALIZER;
pthread_mutex_t recMutex = PTHREAD_RECURSIVE_MUTEX_INITIALIZER_NP;
pthread_mutex_t errchkMutex = PTHREAD_ERRORCHECK_MUTEX_INITIALIZER_NP;
```

Remarque. Le suffixe `_NP` dans les noms de ces constantes symboliques, précise que celles-ci correspondent à des extensions de Posix non portables !

b) Initialisation dynamique

La fonction `pthread_mutex_init()` permet l'initialisation d'une variable Mutex déclarée sans initialisation.

Son prototype est le suivant :

```
int pthread_mutex_init (pthread_mutex_t *mutex, const pthread_mutex_attr *attr);
```

Le second paramètre permet de définir les propriétés du mutex à créer.

En pratique, l'initialisation d'un mutex avec des attributs par défaut, est réalisée comme suit :

```
pthread_mutex_t myMutex;
/* ... */

CHECK_T(pthread_mutex_init(&myMutex, NULL), "pthread_mutex_init()");
```

Cette opération d'initialisation doit être utilisée avant toute autre utilisation de la variable `myMutex`

Ce mutex est créé dans l'état déverrouillé et avec le type *rapide*

(cf. `man pthread_mutex_init`)

2.4 Verrouiller et déverrouiller

Lorsqu'un mutex est créé et initialisé, il peut être utilisé pour verrouiller ou déverrouiller l'accès à une ressource critique.

a) Verrouillage

La fonction ci-dessous :
`int pthread_mutex_lock (pthread_mutex_t *mutex);`
réalise le verrouillage de mutex s'il est libre et provoque la mise en attente du thread appelant, s'il est occupé.
Pour éviter la mise en attente, un thread peut utiliser la fonction `int pthread_mutex_trylock (pthread_mutex_t *mutex);`
qui réalise le verrouillage du mutex s'il est libre, ou retourne le code `EBUSY` s'il est déjà verrouillé.
Dans le cas d'un mutex rapide, son propriétaire qui demande à le verrouiller à nouveau, est lui-même suspendu.

b) Déverrouillage

Seul le thread propriétaire d'un mutex peut le déverrouiller. Il effectue pour cela un appel à la fonction suivante : `int pthread_mutex_unlock (pthread_mutex_t *mutex);`
Lorsque plusieurs threads sont bloqués en attente d'un mutex, seul l'un d'entre-eux est débloqué. Le choix peut dépendre de leur priorité, de leur politique d'ordonnancement, …
Cette stratégie est déterminée par les attributs définis pour la création du mutex.
Lorsque ces attributs ne sont pas spécifiés, l'ordre de déblocage des threads en attente, est l'ordre de leur blocage.

2.5 Supprimer un mutex

La suppression d'un mutex déverrouillé permet de libérer les ressources qu'il utilise. Cette opération est réalisée par :
`int pthread_mutex_destroy (pthread_mutex_t *mutex);`
Dans l'implémentation LinuxThreads, aucune ressource n'est associée à un mutex, aussi `pthread_mutex_destroy()` ne fait rien d'autre que de vérifier si le mutex est déverrouillé.
Cette fonction renvoie le code d'erreur `EBUSY` si le mutex est encore verrouillé.

3. Exemple d'application

3.1 Objectif

Il s'agit d'apporter les modifications au programme précédent, permettant de garantir que le premier thread qui commence l'affichage de son message puisse le faire jusqu'au bout du message et ceci quel que soit celui qui commence, de sorte qu'il n'y ait plus d'entrelacement des affichages.

3.2 Solution

Il faut créer un mutex pour garantir l'accès exclusif à la ressource écran pendant la totalité de l'affichage du message.

Question 1.
Après avoir créé une copie du programme `mutexThread0.c`, nommée `mutexThread1.c`, apporter les modifications permettant de prendre en compte les spécifications ci-dessus.

Chapitre II

Sémaphores

1. Définition et propriétés

Un sémaphore est un objet géré par le cœur d'un système d'exploitation permettant de :

- contrôler l'accès concurrent à des ressources partagées ;
- synchroniser des processus et des threads dans un environnement de programmation concurrente.

Ce concept a été inventé par Edsger Wybe Dijkstra au début des années 1960 pour le système d'exploitation *THE Operating System* de l'Université d'Eindhoven(*Technische Hogeschool Eindhoven*).
Dans le document initial (EWD35), sont introduites l'opération d'initialisation d'un sémaphore ainsi que deux opérations primitives `P()` et `V()` du néerlandais *Proberen* et *Verhogen* (respectivement *tester* et *incrémenter*).
Un sémaphore est défini par :

- un compteur qui détermine le nombre de ressources disponibles ;
- une file de tâches en attente.

L'opération `P(S)` effectuée sur le sémaphore `S` teste la valeur du compteur de `S`.

- S'il est nul, la tâche qui effectue `P(S)` est suspendue et son identifiant est ajouté à la file des tâches en attente de `S` : c'est une opération *bloquante*
- S'il n'est pas nul, sa valeur est décrémentée et la tâche qui effectue cette opération `P(S)` peut accéder à la ressource contrôlée par le sémaphore `S`

L'opération `V(S)` effectuée sur le sémaphore `S` permet d'incrémenter le compteur associé, signifiant ainsi qu'une ressource supplémentaire est à nouveau disponible. Si la file des tâches en attente de `S` n'est pas vide, l'une d'entre-elles, suspendue sur une opération `P(S)`, termine cette opération et accède à la ressource redevenue disponible.
Ces deux opérations `P()` et `V()` sont *atomiques*, c'est-à-dire qu'elles ne peuvent pas être interrompues : il ne peut y avoir préemption d'un processus pendant qu'il exécute l'une ou l'autre de ces opérations, par exemple.
.

2. Sémaphores Posix

Sous UNIX, il existe plusieurs APIs sur les sémaphores :

- l'API des sémaphores `System V`, plus ancienne mais largement disponible sur les systèmes d'exploitation héritiers d'UNIX `System V`, comme `AIX`, `HP-UX`, ainsi que ceux issus de la branche UNIX `BSD`. Cette API peut aussi être utilisée sous `Linux`.

- l'API spécifiée dans la norme Posix sur les extensions temps réel (`IEEE Std.1003.1b-1993`)

La suite de ce document concerne exclusivement les sémaphores Posix, mieux conçus et plus simples d'utilisation que les sémaphores `System V`.
Une introduction sur les sémaphores Posix est accessible en ligne à l'aide de la commande `man sem_overview`
Les deux opérations de base, précédemment notées `P()` et `V()`, correspondent respectivement aux fonctions `sem_wait()` et `sem_post()`
Les types et fonctions associées aux sémaphores Posix commencent par le préfixe `sem_`, d'une manière comparable aux types et fonctions sur les threads qui commencent par `thread_`

2.1 Fichier d'en-tête, type et librairie

Un programme `C` qui utilise des sémaphores doit inclure le fichier qui contient leur définition ;
```
#include <semaphore.h>
```
Ce fichier donne accès à la définition d'un type *opaque* nommé `sem_t` que l'on utilise pour déclarer des variables de type sémaphore.
Le code des fonctions sur les sémaphores est présent dans la libraire pthread. Sur un système LINUX 64 bits on peut s'en rendre compte en exécutant l'une ou l'autre des commandes ci-dessous :
```
nm -D /lib/x86_64-linux-gnu/libpthread.so.0 | grep sem_
objdump  -T /lib/x86_64-linux-gnu/libpthread.so.0 | grep sem_
```

Question 2.

a. Quel est le rôle de la commande `nm` et de son option `-D` ?

b. Quel est le rôle de la commande `objdump` et de son option `-T` ?

Remarque. L'implémentation sous `macOS`, ne prend pas en charge les sémaphores anonymes. Quelques opérations — secondaires — ne sont pas implémentées.

3. Opérations sur sémaphores

Les principales différences entre l'utilisation de sémaphores nommés et de sémaphores anonymes concernent les opérations de création/ouverture ou d'initialisation ainsi que les opérations de fermeture et/ou de suppression.
Les deux sections qui suivent décrivent ces opérations spécifiques alors que la troisième section décrit les opérations communes.
On notera cependant que les opérations sur sémaphores nommés ou anonymes …

- sont des fonctions de bibliothèque — et non des appels système — décrites dans la section 3 des manuels UNIX et dérivés ;
- respectent la convention de résultat définie dans la norme `Posix 1003.1b` : elles retournent -1 en cas d'échec, et positionnent le code de l'erreur dans la variable `errno`. Seule la fonction `sem_open()` retourne un résultat de nature différente.
- sont définies avec un paramètre de type pointeur pour désigner le sémaphore utilisé, comme dans cet exemple : `int sem_post(sem_t *sem);` C'est par conséquent, l'adresse d'un objet sémaphore créé et initialisé que l'on doit fournir à ces opérations (hors opération de création/ouverture) ;

3.1 Opérations spécifiques sur sémaphores nommés

a) `sem_open()`

Il y a deux manières distinctes d'utiliser cette fonction :

1. l'une, pour utiliser un sémaphore préalablement créé par un autre processus, par exemple ;
2. l'autre pour créer un nouveau sémaphore.

Dans les deux cas, il est utile d'inclure quelques fichiers d'entête supplémentaires, pour définir les modes d'ouverture ainsi que les permissions, à l'aide de noms symboliques :

```
#include <fcntl.h>
#include <sys/stat.h>
```

Dans ces deux usages, le résultat retourné en cas de succès est l'adresse du sémaphore, c'est-à-dire un pointeur d'objet sémaphore.
En cas d'échec, c'est la valeur `SEM_FAILED` qui est retournée, et le code de l'erreur est rangé dans la variable `errno`
Il est par conséquent utile de créer et d'utiliser une variante de la macro `CHECK()` :

```
#define CHECK_IFFAILED(sts, msg)     \
    if (SEM_FAILED == (sts) )        \
    {                                \
        perror(msg);                 \
        exit(EXIT_FAILURE);          \
    }
```

Remarque. Cette opération établit un **lien** système entre l'objet sémaphore et son fichier spécial.

1. **Ouverture d'un sémaphore existant**

   ```
   sem_t *sem_open(const char *name, int oflag);
   ```

   - `name` est le nom du sémaphore. Ce nom peut commencer par un *backslash*, mais ce caractère est interdit dans la suite du nom. Qu'il soit présent ou non, le fichier spécial est créé dans le dossier `/dev/shm`
   - `oflag` définit le mode d'accès au sémaphore. On exprime généralement le mode d'accès à l'aide de constantes symboliques définies dans `fcntl.h`, comme `O_RDONLY`, `O_WRONLY`, `O_RDWR`.
     En pratique, pour l'accès à un sémaphore existant, ce paramètre vaut zéro.

   Exemple.

   ```
   sem_t * sem_test;

   /* ... */
   CHECK_IFFAILED(sem_test = sem_open("mySem", 0), "sem_open() ");
   ```

   Les principales causes d'échec sont les suivantes :

   ENOENT : le sémaphore désigné par le nom donné, n'existe pas ;

   EINVAL : le nom du sémaphore n'est invalide ;

EACCES : le sémaphore existe mais les permissions de l'appelant sont insuffisantes pour l'ouvrir.

2. **Création d'un nouveau sémaphore**

   Cette opération est réalisée à l'aide de la fonction ci-dessous :

   ```
   sem_t *sem_open(const char *name, int oflag, mode_t mode, unsigned int value);
   ```

   Le second paramètre (`oflag`) prend dans ce cas la valeur `O_CREAT` et le troisième paramètre (`mode`) permet de définir les permissions attribuées au sémaphore créé.

   On peut définir les permissions à l'aide d'une valeur numérique souvent exprimée en octal, comme dans cet exemple :

   La permission `0640` sur un fichier, donne :

   - l'accès en lecture et en écriture à l'utilisateur ;
   - uniquement l'accès en lecture aux utilisateurs du groupe ;
   - aucun accès aux autres utilisateurs

   On peut aussi exprimer les permissions en utilisant une combinaison de constantes symboliques définies dans `fcntl.h`, comme par exemples :

   S_IWUSR : qui donne l'accès en lecture pour l'utilisateur ;

   S_IWRITE : qui donne l'accès en écriture pour l'utilisateur ;

   S_IXUSR : qui donne l'accès en exécution pour l'utilisateur ;

   S_IRWXU : qui donne les trois accès précédents à l'utilisateur.

   D'autres noms de constantes symboliques existent définir les permissions du groupe et des autres utilisateurs.

   Exemple.

   `S_IWUSR | S_IWRITE | S_IRGRP` qui donne à l'utilisateur les accès en lecture et en écriture, et l'accès en lecture au groupe.

   Le dernier paramètre de la fonction `sem_open()` permet de définir la valeur initiale du compteur associé au sémaphore.

   Remarque.
   Avec le mode `O_CREAT`, il y a création d'un sémaphore nommé s'il n'existe pas et ouverture s'il existe. On peut modifier ce comportement en utilisant le mode combiné `O_CREAT | O_EXCL` qui génère une erreur lorsque le sémaphore à créer, existe déjà.

Exemple.

```
sem_t * sem_test;

/* ----- */
CHECK_IFFAILED(sem_test = sem_open("myTest", O_CREAT, S_IRWXU, 1),
    "sem_open()");
```

Enfin, il faut noter que les processus enfants héritent de l'accès au sémaphore qu'avait préalablement créé ou ouvert le processus parent.

b) `sem_close()`

Cette fonction libère les ressources du système affectées à la gestion d'un sémaphore.

```
int sem_close(sem_t *sem);
```

Cette opération supprime également le lien qui existait entre le sémaphore et fichier spécial.

c) `sem_unlink()`

Cette fonction est une demande de suppression du pseudo-fichier associé à un sémaphore nommé ; la suppression effective n'ayant lieu dès qu'à l'instant où le dernier processus l'utilisant effectue lui-même un appel à cette fonction.

```
int sem_unlink(const char *name);
```

On notera que cette fonction utilise le nom du sémaphore tel qu'il a été utilisé dans l'opération `sem_open()`

La suppression effective du fichier spécial associé, est réalisé dès que le compteur de liens sur le fichier spécial, a atteint la valeur nulle.

3.2 Opérations spécifiques sur sémaphores anonymes

a) `sem_init()`

Avant d'utiliser un sémaphore anonyme, celui-ci doit être initialisé à l'aide de la fonction :

```
int sem_init(sem_t *sem, int pshared, unsigned int value);
```

- Le premier paramètre est un pointeur qui désigne le sémaphore à initialiser ;
- Le second paramètre vaut 0 pour indiquer que le sémaphore est partagé entre threads. S'il a une valeur non nulle, il est partagé entre processus. Dans ce dernier cas, le sémaphore doit être défini dans une zone de mémoire partagée, créée par une opération `shm_open()`
- Le troisième paramètre détermine la valeur initiale du compteur associé au sémaphore ; ce doit être une valeur positive ou nulle.

Cette opération qui ne doit être effectuée qu'une seule fois pour un sémaphore donné, est généralement réalisée dans le thread principal, avant que ne soient créés les threads utilisateurs.

Exemple.

```
sem_t semEcran; /* variable globale */
/* ... */
CHECK(sem_init(&semEcran, 0, 1), "sem_init()");
```

Le compteur associé au sémaphore `semEcran` est initialisé à 1 : ce sémaphore a alors une fonction comparable au mutex pthread, utilisé dans le chapitre précédent.

b) `sem_destroy()`

La fonction `sem_destroy()` permet de supprimer un sémaphore préalablement initialisé à l'aide de l'opération `sem_init()`

```
int sem_destroy(sem_t *sem);
```

En pratique, le thread principal qui a initialisé un sémaphore, a généralement la responsabilité de le supprimer dès qu'il n'est plus utilisé.

Exemple.

```
CHECK(sem_destroy(&semEcran), "sem_destroy()");
```

3.3 Opérations communes aux deux types de sémaphores

a) sem_wait()

La fonction `sem_wait()` correspond à la primitive `P()` décrite précédemment.

`int sem_wait(sem_t *sem);`

Elle permet d'accéder à la ressource protégée par le sémaphore si son compteur est positif, ou suspend le thread ou le processus lorsque son compteur est nul. L'accès à la ressource décrémente son compteur.

Exemple.

```
CHECK(sem_wait(&semEcran), "sem_wait()");
```

b) sem_post()

La fonction `sem_post()` correspond à la primitive `V()` décrite précédemment.

`int sem_post(sem_t *sem);`

Elle sert à restituer un exemplaire de la ressource protégée par le sémaphore, provoquant ainsi l'incrémentation du compteur qui lui est associé. Cet exemplaire devenu disponible, peut être pris par un des threads ou un des processus qui l'attendent.

Exemple.

```
CHECK(sem_post(&semEcran), "sem_post()");
```

c) sem_trywait()

`int sem_trywait(sem_t *sem);` C'est une variante non bloquante de `sem_wait()`. Si le compteur associé au sémaphore désigné par `sem` n'est pas nul, alors `int sem_trywait()` se comporte exactement comme `int sem_wait()`

Si le compteur du sémaphore est à 0, alors cette fonction se termine immédiatement en retournant la valeur -1 et en positionnant la variable `errno` à `EAGAIN`.

Ne pas utiliser la macro `CHECK()` avec cette fonction.

d) sem_timedwait()

`int sem_timedwait(sem_t *sem, const struct timespec *abs_timeout);`

Cette fonction se comporte comme la fonction `sem_wait()` à la différence qu'un instant limite d'attente est spécifié.

Lorsque l'instant limite est atteint, cette fonction se termine et retourne la valeur -1. Le code d'erreur `ETIMEDOUT` est rangé dans la variable `errno`.

L'argument `timeout` est un pointeur sur une structure qui spécifie un temps absolu en secondes et nanosecondes depuis 1er `janvier 1970 à 00:00:00 (UTC)`.

Cette structure est définie de la manière suivante :

```
struct timespec {
   time_t tv_sec;      /* Secondes */
   long   tv_nsec;     /* Nanosecondes [0 .. 999999999] */
};
```

Ne pas utiliser la macro `CHECK()` avec cette fonction.

e) `sem_getvalue()`

```
int sem_getvalue(sem_t *sem, int *sval);
```

Cette fonction range à l'adresse transmise dans `sval`, la valeur actuelle du compteur associé au sémaphore désigné par `sem`

4. Exercice

Question 3.

Créer une copie du programme `mutexThread1.c` qui sera nommée `mutexThread2.c`, puis lui apporter les modifications nécessaires pour remplacer l'usage d'un pthread mutex par celui d'un sémaphore Posix.

5. Sémaphores vs mutex pthread

L'exercice précédent montre que l'on peut faire avec un sémaphore ce que fait un mutex pthread.
Il y a cependant quelques différences notables :

1. un sémaphore permet de protéger à l'accès à une ressource disponible en plusieurs exemplaires. On le désigne comme étant un sémaphore à compte, alors qu'on peut assimiler le pthread mutex à un sémaphore d'exclusion mutuelle : la ressource n'est présente qu'en un seul exemplaire et elle n'a, à un instant donné au plus qu'un seul propriétaire.

2. dans le cas d'un pthread mutex, seul le thread qui a verrouillé le mutex, peut le déverrouiller.

   On peut avoir un thread ou un processus qui effectue une opération `sem_wait()` et un autre thread ou processus qui réalise l'opération `sem_post()` sur le même sémaphore.

3. Les sémaphores, notamment les sémaphores nommés, peuvent être utilisés par des processus n'ayant aucun lien de parenté père-fils.

   Les mutex de threads Posix, ne permettent pas cet usage, sauf à les créer dans une zone mémoire partagée Posix (cf. `man shm_overview`)

Chapitre III

Exercice de synthèse

1. Présentation et objectifs

1.1 Objectif

Il s'agit d'implémenter de manière progressive, le problème classique des **lecteurs et rédacteurs**.
Un rédacteur est chargé de ranger dans un buffer circulaire de `NBMAX_BUFFER` compartiments, des valeurs successives comprises entre 0 et `MAX_VALUE - 1` : la première valeur est rangée dans le premier compartiment, la seconde dans la suivante, et lorsque le dernier compartiment est rempli, la prochaine valeur sera rangée dans le premier compartiment, et ainsi de suite. Un rédacteur ne peut remplir un compartiment que s'il est vide ; un compartiment est considéré comme vide si son contenu a été lu par tous les lecteurs.
Un lecteur est chargé de lire toutes les valeurs écrites dans le buffer circulaire par le ou les rédacteurs dans l'ordre avec lequel elles ont été écrites.
La valeur `OVER` est la dernière valeur écrite par un rédacteur. Le lecteur s'arrête dès qu'il a lu cette valeur.
L'objectif final de cet exercice est la réalisation d'un programme permettant :

- à n rédacteurs de se partager les écritures dans un buffer circulaire ;
- à m lecteurs de lire chacun, chaque donnée écrite dans ce buffer.

Cette réalisation sera effectuée en plusieurs étapes :

1. la version « un rédacteur et un lecteur » ;
2. la version « un rédacteur et m lecteurs » ;
3. la version « n rédacteurs et m lecteurs » ;

1.2 Spécifications détaillées

a) Constantes symboliques

Le programme à réaliser utilisera les définitions de constantes symboliques suivantes :

```
#define NBMAX_BUFFER    3
#define MAX_VALUE       10
#define OVER           (-1)

#define TEMPO_1ms       1000000L
#define MAXLEN          255
```

b) Bufffer circulaire

Le buffer circulaire utilisé par le(s) rédacteur(s) et le(s) lecteur(s) est défini du type suivant :

```
typedef struct {
    int buffer[NBMAX_BUFFER]; /* la zone de stockage                          */
    /*  ............................ à compléter  ........................... */
} circBuffer_t;
```

Il doit être déclaré de manière globale, afin de pouvoir être initialisé dans le thread principal et utilisé dans les fonctions qui implémentent les threads lecteur et rédacteur.

`circBuffer_t buf;`

c) Affichage des messages

Lecteur(s) et rédacteur(s) signalent les traitements réalisés en affichant des messages à l'écran, à l'aide de la fonction ci-dessous ;

```
void putLine (const char *s)
{
    for ( ; *s ==  ' '; s++) printf("%c", *s);
    fflush(stdout);
    for ( ; *s != '\0'; s++) {
        tempo(0, 100);
        printf("%c", *s);
        fflush(stdout);
    }
}
```

d) Temporisation

Dans la fonction `putLine()` ci-dessus, une attente d'une durée tirée aléatoirement dans l'intervalle $[0, 100]$ est effectuée entre l'affichage de deux caractères successifs d'un message — hors caractères d'espacement en tête de message —
Cette attente est produite à l'aide de la fonction `tempo()`

```
void tempo(int min, int max)
{

    struct timespec delai = { .tv_sec = 0, .tv_nsec = 0};
    delai.tv_nsec = TEMPO_1ms * (min + rand() % (max - min + 1));
    nanosleep(&delai, NULL);
}
```

e) Rédacteur

La version de base à compléter de la fonction qui implémente un rédacteur, se présente comme suit :

```
void  redacteur(long no)
{
    int i = 0, n = 0;
    char line[MAXLEN + 1];

    printf("Lancement du rédacteur %ld d'id = 0x%08lx\n",
           no, (long)pthread_self());

    while  ( n != OVER ) {
        if (n == MAX_VALUE)
            buf.buffer[i] = n = OVER;
        else
```

```
            buf.buffer[i] = n++;
        sprintf(line,"Le rédacteur n°%ld écrit %d dans le buffer %d\n",
                no, buf.buffer[i] , i);
        putLine(line);
        i++;
        if (i == NBMAX_BUFFER) i = 0;
    }
    return NULL;
}
```

Après s'être identifié par son numéro passé en argument et par son identifiant de thread, le rédacteur range des valeurs entières successives dans de compartiments successifs du buffer circulaire.
Lorsque la valeur maximale est atteinte, le rédacteur range la valeur `OVER` et se termine.

f) Lecteur

La version de base à compléter, de la fonction qui implémente un lecteur, se présente comme suit :

```
void  lecteur(long no)
{
    int i = 0, n = 0;
    char line[MAXLEN + 1];

    printf("Lancement du lecteur %ld d'id   = 0x%08lx\n",
           no, (long)pthread_self());

    while ( n != OVER ) {
        n = buf.buffer[i];
        sprintf(line,"%*sLe lecteur n°%ld lit %d dans le buffer %d\n",
                5 * (int)(no + 1), "  ", no,   n, i);
        putLine(line);
        i++;
        if (i == NBMAX_BUFFER) i = 0;
    }
    return NULL;
}
```

Le lecteur, comme le rédacteur, s'identifie lors de son lancement ; il affiche son numéro présent dans le paramètre de sa fonction et son identifiant, puis lit séquentiellement le contenu de chaque compartiment du buffer circulaire et affiche la valeur lue.
Il s'arrête dès qu'il a lu la valeur `OVER`

2. Un rédacteur et un lecteur

Dans cette première version, on considère qu'il n'y a qu'un seul rédacteur et un seul lecteur.
Le code de la version de base est disponible sur la plateforme **Moodle** de cet enseignement : c'est le fichier `lectRedac0.c`

2.1 Test de la version initiale

Question 4.

Générer le programme exécutable `lectRedac0` et l'exécuter.
Quel(s) problème(s) observe-t-on ?

a) Test n° 3

Générer à nouveau ce programme en utilisant d'option `-DNOWRITER` sur la ligne de commande de `gcc`

Question 5.

a. Que signifie cette option ?

b. Quelle incidence a-t-elle pour le programme généré ?

2.2 Accès concurrent au buffer circulaire

Les tests qui viennent d'être réalisés montrent que le rédacteur ne se préoccupe pas de vérifier qu'un compartiment est vide pour le remplir. De même le lecteur lit le contenu d'un compartiment du buffer circulaire, alors que rien n'y a été inscrit.
La résolution de ce problème passe par l'utilisation de deux sémaphores à compte :

- `semBufferVide` qui permet de suspendre le processus rédacteur lorsque tous les compartiments sont occupés ; le compteur qui lui est associé, symbolise le nombre de compartiments vides
- `semBufferVide` qui permet de suspendre le processus lecteurs lorsque tous les compartiments sont vides ; le compteur qui lui est associé, symbolise le nombre de compartiments pleins.

Ces deux sémaphores étant directement associés à l'utilisation du buffer circulaire, il est logique d'en faire deux attributs supplémentaires de la structure de buffer circulaire.

```
typedef struct {              /* buffer circulaire d'entiers                */
    int buffer[NBMAX_BUFFER]; /* la zone de stockage                        */
    sem_t semBufferPlein;     /* nombre de compartiments à lire             */
    sem_t semBufferVide;      /* nombre de compartiments à remplir          */
    /*  ........................... à compléter  ........................... */
} circBuffer_t;
```

a) Initialisation des sémaphores

Leur initialisation sera réalisée dans la fonction `init()` en posant les questions suivantes :
Au lancement du programme :

- combien y-a-t-il de compartiments vides dans le buffer circulaire ?
- combien y-a-t-il de compartements pleins ?

Question 6.
Compléter en conséquence la fonction `init()`

b) Suppression des sémaphores

La suppression des sémaphores doit avoir lieu après leur usage dans les threads lecteur et rédacteur. Sachant qu'ils ont été initialisés dans le thread principal, il est logique qu'ils soient supprimés également dans la fonction `main()`.

Question 7.
Compléter la fonction principale de manière à supprimer les sémaphores devenus inutiles

c) Test n° 2

Générer à nouveau ce programme en utilisant d'option `-DNOREADER` sur la ligne de commande de `gcc`

Question 8.
Le comportement du programme ainsi obtenu, est-il conforme aux spécifications ?

d) Test n° 3

Générer à nouveau ce programme en utilisant d'option `-DNOWRITER` sur la ligne de commande de `gcc`

Question 9.
Le comportement du programme ainsi obtenu, est-il conforme aux spécifications ?

---

3. Un rédacteur et *m* lecteurs

3.1 Spécifications

Dans cette version une valeur inscrite dans un compartiment doit être lue par chaque lecteur : ce compartiment sera considéré comme libre (vide) lorsque tous les lecteurs auront lu la valeur contenue. Tant que ceci n'est pas fait, ce compartiment est considéré comme rempli (occupé).
Naturellement quand un rédacteur vient de remplir un compartiment vide, celui-ci est considéré comme plein pour chaque lecteur.
Le reste des spécifications est inchangé par rapport au cas traité précédemment.

3.2 Modifications à apporter

On fixera à 3 le nombre de lecteurs traités dans le programme

```
#define NBLECTEURS  3
```

Créer une copie du programme précédent, nommée `lectRedac2.c` et lui apporter les modifications nécessaires à la prise en compte des spécifications supplémentaires, rappelées ci-dessus :
Ce programme doit permettre de produire un résultat comparable à celui-ci :

```
$ ./lectRedac2
Lancement du programme d'id   = 0x7f5e76e93700
Lancement du lecteur 0 d'id   = 0x7f5e766c9700
Lancement du lecteur 1 d'id   = 0x7f5e75ec8700
Lancement du lecteur 2 d'id   = 0x7f5e756c7700
Lancement du rédacteur 0 d'id = 0x7f5e74ec6700
Le rédacteur n°0 écrit 0 dans le buffer 0
Le rédacteur n°0 écrit 1 dans le buffer 1
     Le lecteur n°0 lit 0 dans le buffer 0
          Le lecteur n°1 lit 0 dans le buffer 0
               Le lecteur n°2 lit 0 dans le buffer 0
Le rédacteur n°0 écrit 2 dans le buffer 2
     Le lecteur n°0 lit 1 dans le buffer 1
          Le lecteur n°1 lit 1 dans le buffer 1
               Le lecteur n°2 lit 1 dans le buffer 1
Le rédacteur n°0 écrit 3 dans le buffer 0
     Le lecteur n°0 lit 2 dans le buffer 2
          Le lecteur n°1 lit 2 dans le buffer 2
               Le lecteur n°2 lit 2 dans le buffer 2
```

```
Le rédacteur n°0 écrit 4 dans le buffer 1
     Le lecteur n°0 lit 3 dans le buffer 0
     Le lecteur n°0 lit 4 dans le buffer 1
               Le lecteur n°2 lit 3 dans le buffer 0
...
Le rédacteur n°0 écrit -1 dans le buffer 1
     Le lecteur n°0 lit 9 dans le buffer 0
          Le lecteur n°1 lit 9 dans le buffer 0
               Le lecteur n°2 lit 9 dans le buffer 0
     Le lecteur n°0 lit -1 dans le buffer 1
          Le lecteur n°1 lit -1 dans le buffer 1
               Le lecteur n°2 lit -1 dans le buffer 1
```

Question 10.

Présenter les modifications apportées au programme précédent.

3.3 Tests

Question 11.

a. Quel résultat doit produire la commande `./lectRedac2 | wc -l` ?

b. Après avoir redirigé le résultat de programme dans le fichier `lectRedac2.txt`,

Que permet de vérifier la commande

```
for i in $(seq 0 2); do grep "lecteur n°"$i lectRedac2.txt | sed "s/^ *//"; done
```

c. Quelle commande faut-il exécuter pour n'afficher que les opérations faites par le rédacteur, présentes dans le fichier `lectRedac2.txt`

4. *n* rédacteurs et *m* lecteurs

4.1 Spécifications

Dans cet exercice, plusieurs rédacteurs se partagent les opérations d'écriture de valeurs successives dans des compartiments successifs du buffer circulaire.
Ces nouvelles spécifications sont sans incidence sur le fonctionnement des lecteurs qui, comme dans l'exercice précédent, doivent lire chaque valeur écrite dans ce buffer circulaire.

4.2 Modifications à apporter

Créer une copie du programme précédent, nommée `lectRedac3.c`.
Dans cet exercice, on fixe le nombre de rédacteurs à 2.

```
#define NBREDACTRS 2
```

La principale modification concerne la définition de la structure de buffer circulaire :

- deux rédacteurs ne doivent pas écrire la même valeur ;
- deux rédacteurs ne peuvent simultanément utiliser le même compartiment du buffer circulaire.

a) Nouvelle définition du buffer circulaire

La conséquence de ces contraintes est l'ajout dans la structure du buffer circulaire de :

- la prochaine valeur à écrire dans un compartiment ;
- l'indice du prochain compartiment dans lequel écrira un rédacteur ;
- un mutex, pour garantir que deux rédacteurs ne puissent simultanément accéder aux deux informations précédentes.

Question 12.
Apporter les modifications signalées ci-dessus, à la déclaration de la structure `circBuffer_t`.

b) Modification du rédacteur

Le premier des rédacteurs qui écrit la valeur -1, met fin aux opérations d'écriture, y compris des autres rédacteurs. Autrement dit, quel que soit le nombre de rédacteurs, la valeur -1 n'est écrite qu'une seule fois.

Question 13.
Apporter les modifications à la fonction `redacteur()` de façon à prendre en compte les nouvelles spécifications.

4.3 Tests

Question 14.
Générer le programme `lectRedac3` et l'exécuter en redirigeant la sortie vers le fichier `lectRedac3.txt`

1. Quelle est la commande permettant d'afficher toutes les lignes signalant l'écriture d'une valeur.
2. Comment afficher le nombre de lignes signalant une écriture d'une valeur.
3. Les résultats sont-ils conformes aux résultats attendus ?

Bibliographie

[1] W. Richard Stevens, Stephen A. Rago
Advanced Programming in the UNIX Environment, Addison-Wesley, Third Edition, 2013.

[2] Robert Love
Linux System Programming, O'Reilly, 2007.

[3] Christophe Blaess
Programmation système en C sous Linux, Eyrolles, 2e édition, 2005.

[4] Patrick Cegielski
Conception de systèmes d'exploitation : Le cas Linux, Eyrolles, 2e édition, 2004.

[5] Vincent Lozano
UNIX : Pour aller plus loin avec la ligne de commande
In Libro Veritas, Framabook, 2010
(Téléchargeable ici)

[6] Mendel Cooper, Linux Documentation Project
Advanced Bash-Scripting Guide

- Version anglaise en ligne ;
- Traduction en français en ligne.

[7] Robert Mecklenburg
Managing Projects with GNU Make, O'Reilly, 3rd edition, 2004
(en ligne sur le projet Open Books)

[8] Richard M. Stallman and the GCC Developer Community
Using the GNU Compiler Collection, GNU Press

- Version anglaise en ligne
- Version anglaise au format pdf

[9] O. Bonaventure, G. Detal, C. Paasch
Cours Systèmes Informatiques (2014) de l'Université catholique de Louvain

- Version en ligne
- Version au format pdf